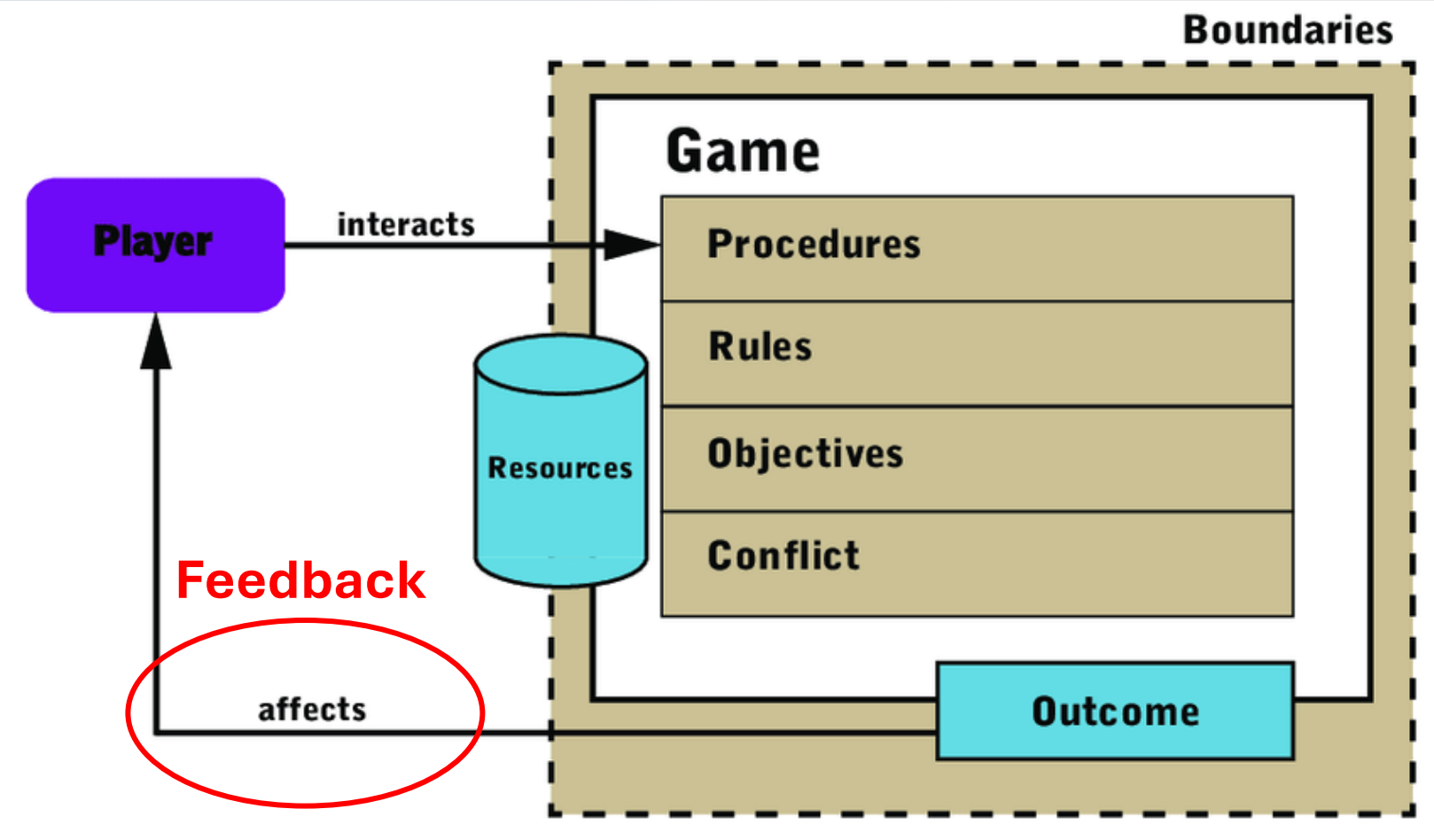
Feedback Design : 2D Platformer Example

Game Design A.A. 2024/25

Stefano Calzolari
stefano.calzolari@polito.it

Feedback ed *Elementi Formali*



MSc Thesis,

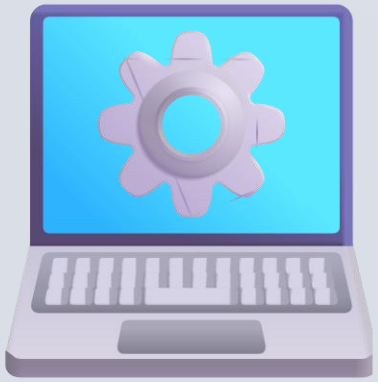
Kathrin Gerling


User Action
State Change
Feedback
Call to Action


User Action
Call to Action
State Change
Feedback

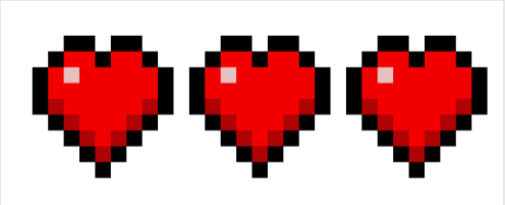
Stato di Gioco

State Change

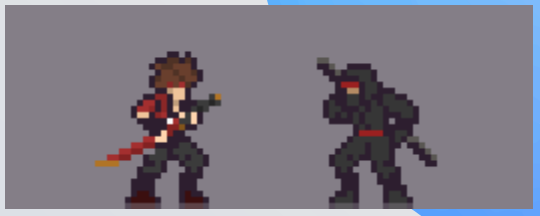


🔢 Variabili

❗ Eventi







Progettare il Feedback - 1

🔢 **Variabili**

❗ **Eventi**

Il giocatore deve conoscere questa **variabile** o **evento**?

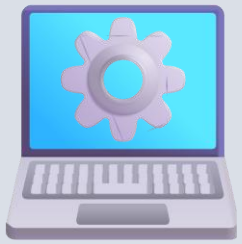
A che **livello di dettaglio**?

Si può rappresentare nello **spazio**?

Si può rappresentare **diegeticamente**?

Progettare il Feedback - 2

State Change



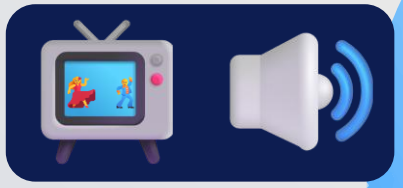


🔢 Variabili

❗ Eventi



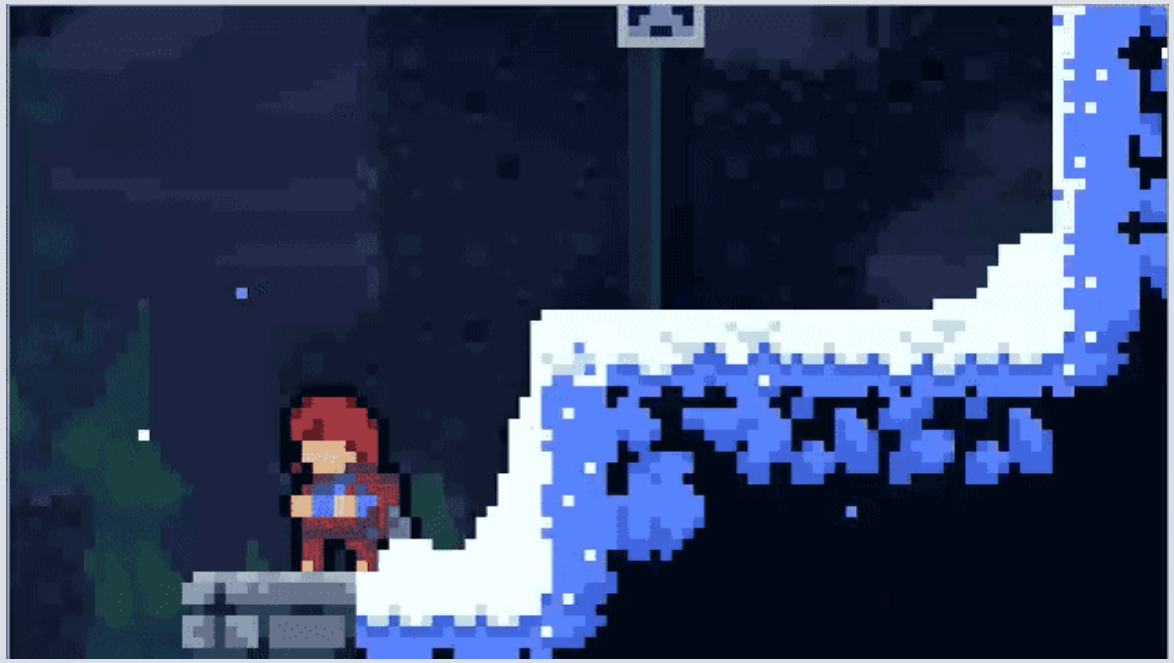
Feedback



Platformer 2D: Meccaniche Principali



Platformer 2D: Meccaniche Principali

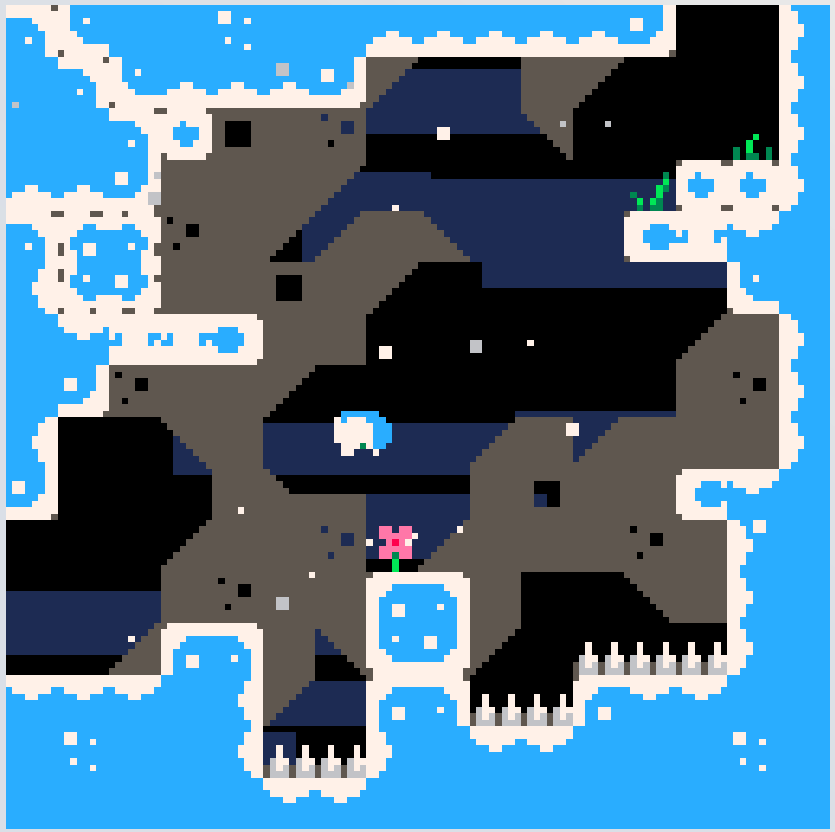


Sistema di
Movimento



Sistema di
Piattaforme e **Ostacoli**

Caso studio: Celeste Pico-8

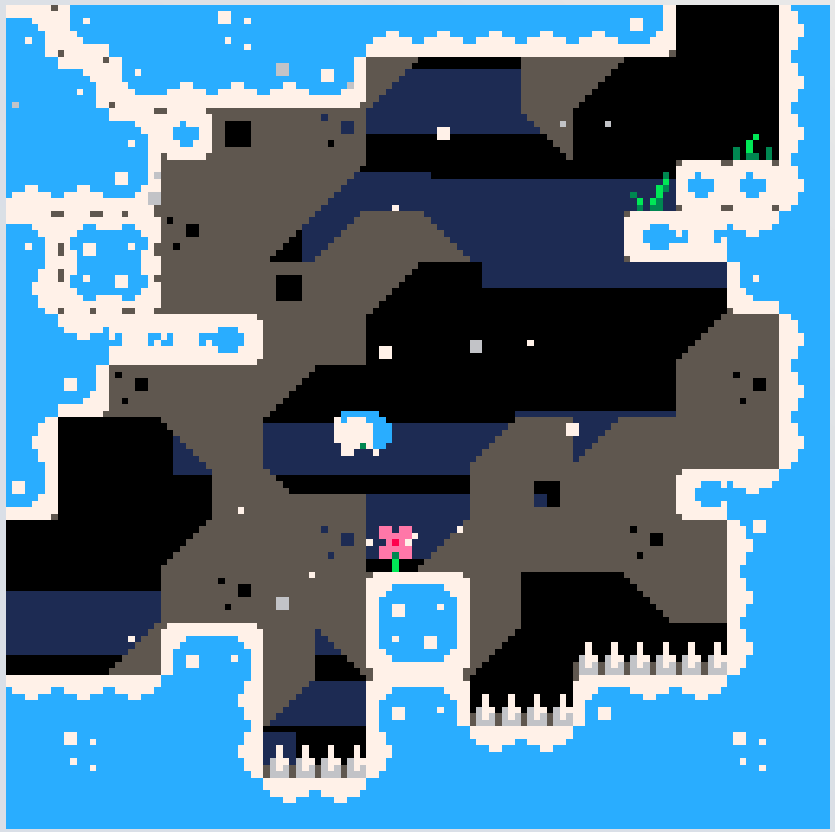


Celeste (Pico-8)

Scomposizione Movimento

Move

Dash



Jump (and Fall)

Slide

Celeste (Pico-8)

Movimento : Analisi Feedback

Move	**Jump (and Fall)**
Dash	**Slide**

Movimento : Analisi Feedback

Move

- Sprite Flipbook
- Flip direction
- Hair movement

Jump (and Fall)

Dash

Slide

Movimento : Analisi Feedback

Move

- Sprite Flipbook
- Flip direction
- Hair movement

Jump (and Fall)

- “InAir” Sprite
- Smoke (jump)
- Smoke (fall)

Dash

Slide

Movimento : Analisi Feedback

Move

- Sprite Flipbook
- Flip direction
- Hair movement

Jump (and Fall)

- “InAir” Sprite
- Smoke (jump)
- Smoke (fall)

Dash

- Smoke trail
- Screen shake
- Hair color

Slide

Movimento : Analisi Feedback

Move

- Sprite Flipbook
- Flip direction
- Hair movement

Jump (and Fall)

- “InAir” Sprite
- Smoke (jump)
- Smoke (fall)

Dash

- Smoke trail
- Screen shake
- Hair color

Slide

- “Slide” Sprite
- Smoke

Movimento : Analisi Feedback

Move

- Sprite Flipbook
- Flip direction
- Hair movement

Jump (and Fall)

- “InAir” Sprite
- Smoke (jump)
- Smoke (fall)

Dash

- Smoke trail
- Screen shake
- Hair color

Slide

- “Slide” Sprite
- Smoke

Source Code

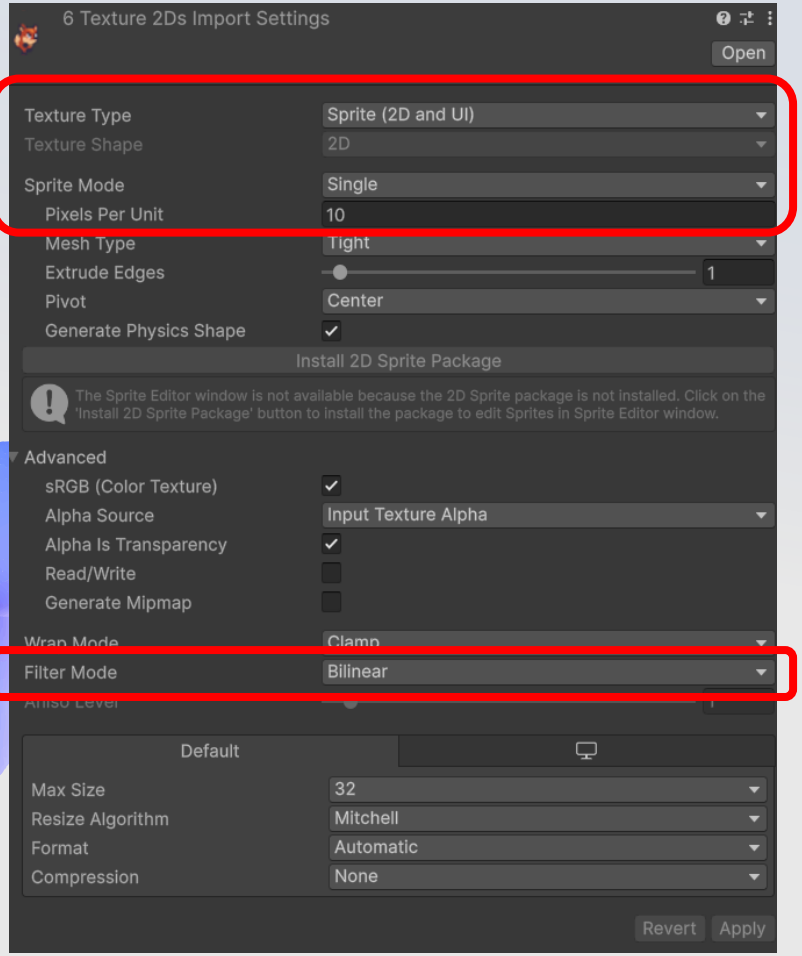
Movimento: Implementazione Feedback

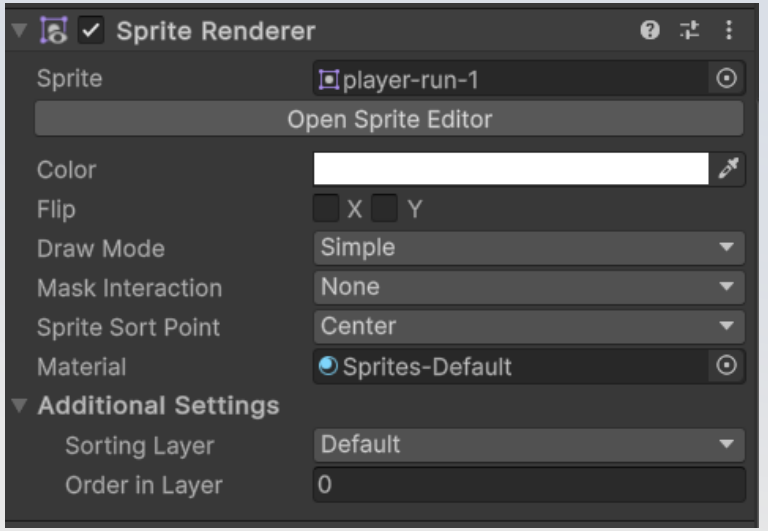


Animazioni Movimento

Foxy Animation Package

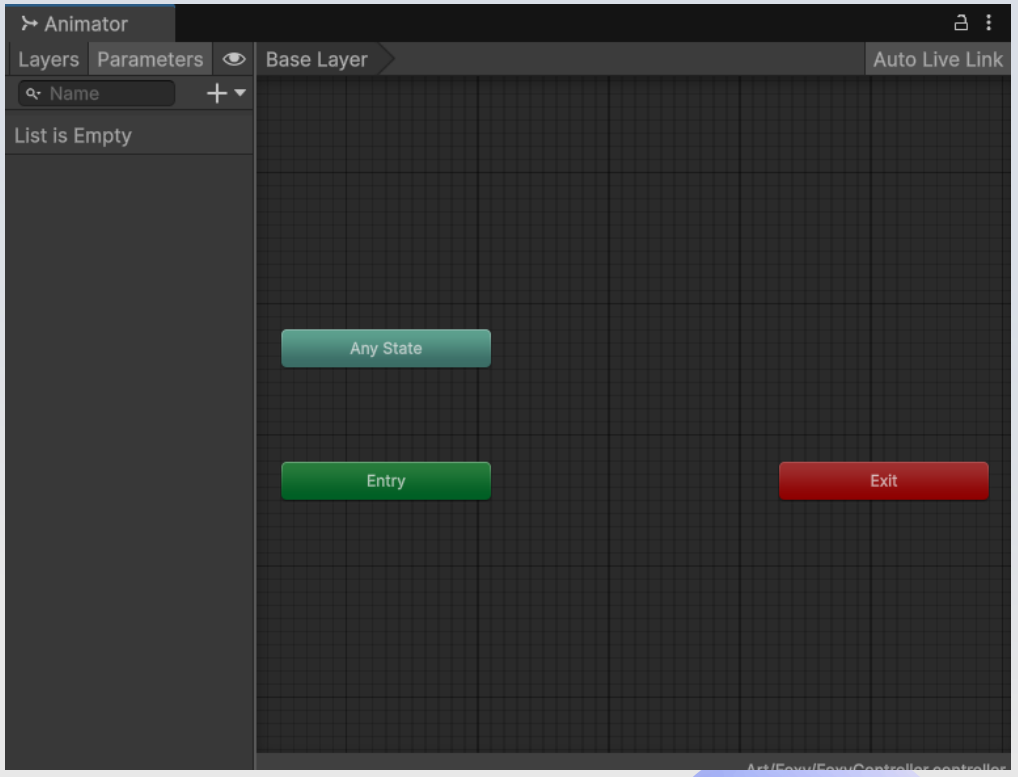
Step 1 – Sprites and Renderer



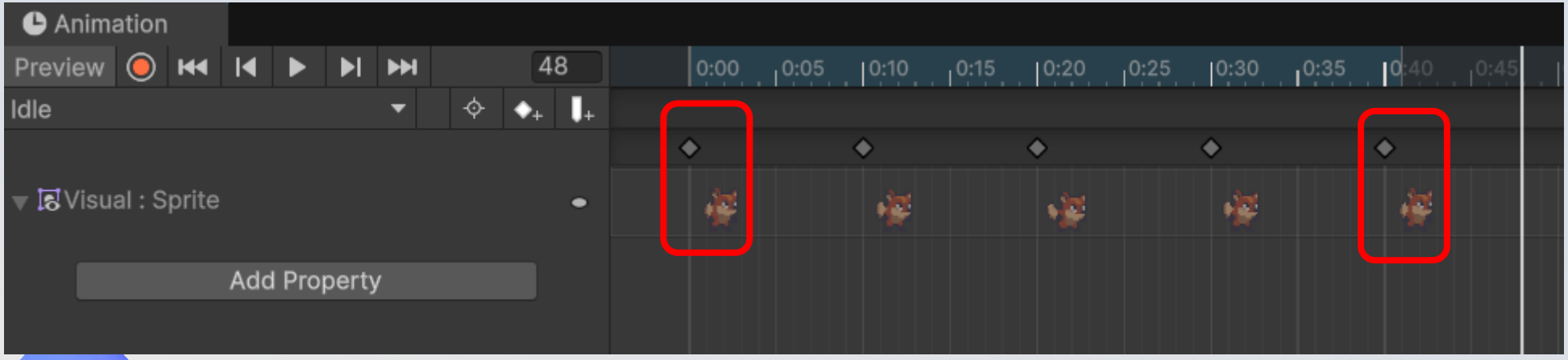


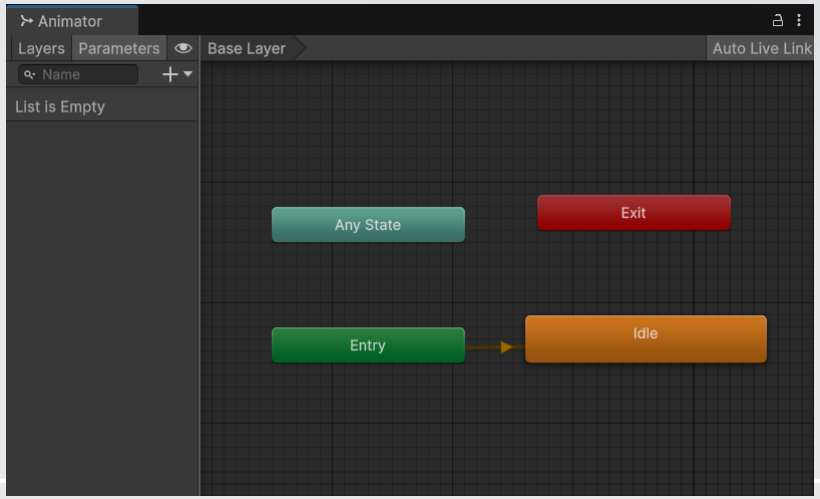
Step 2 – Animator and AnimController



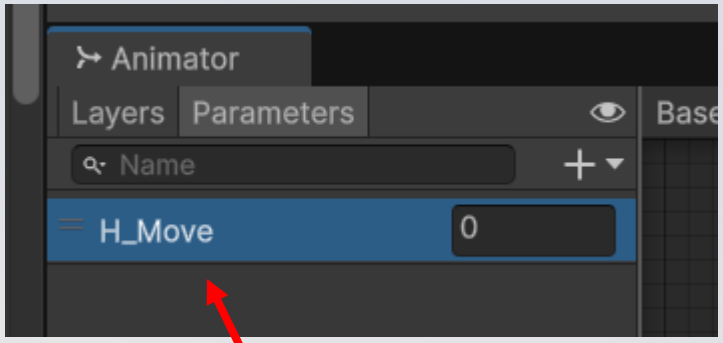


Step 3 – AnimationClip (Flipbooks)





Step 4 – Parameters (Vars and Events)




```
private void Awake()
{
    Assert.IsNull(instance);
    instance = this;

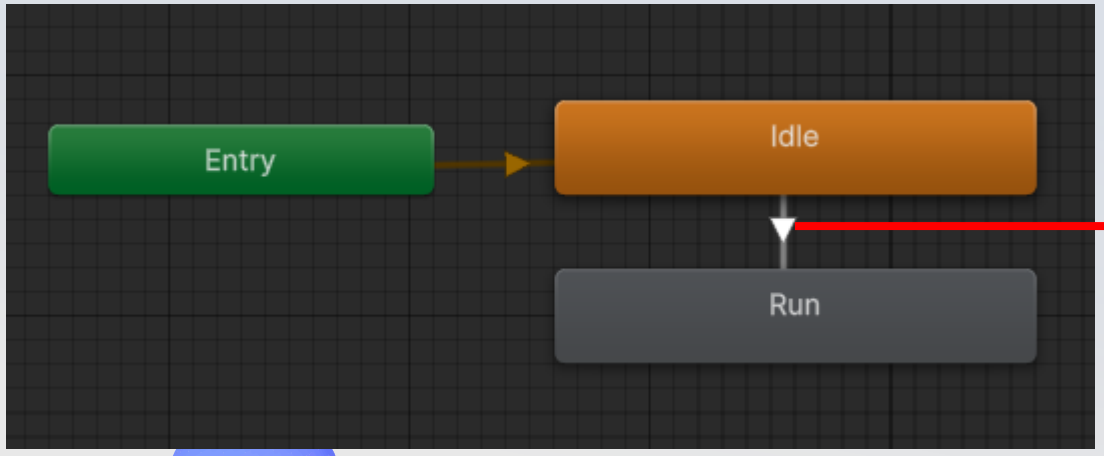
    anim = this.GetComponentInChildren<Animator>();

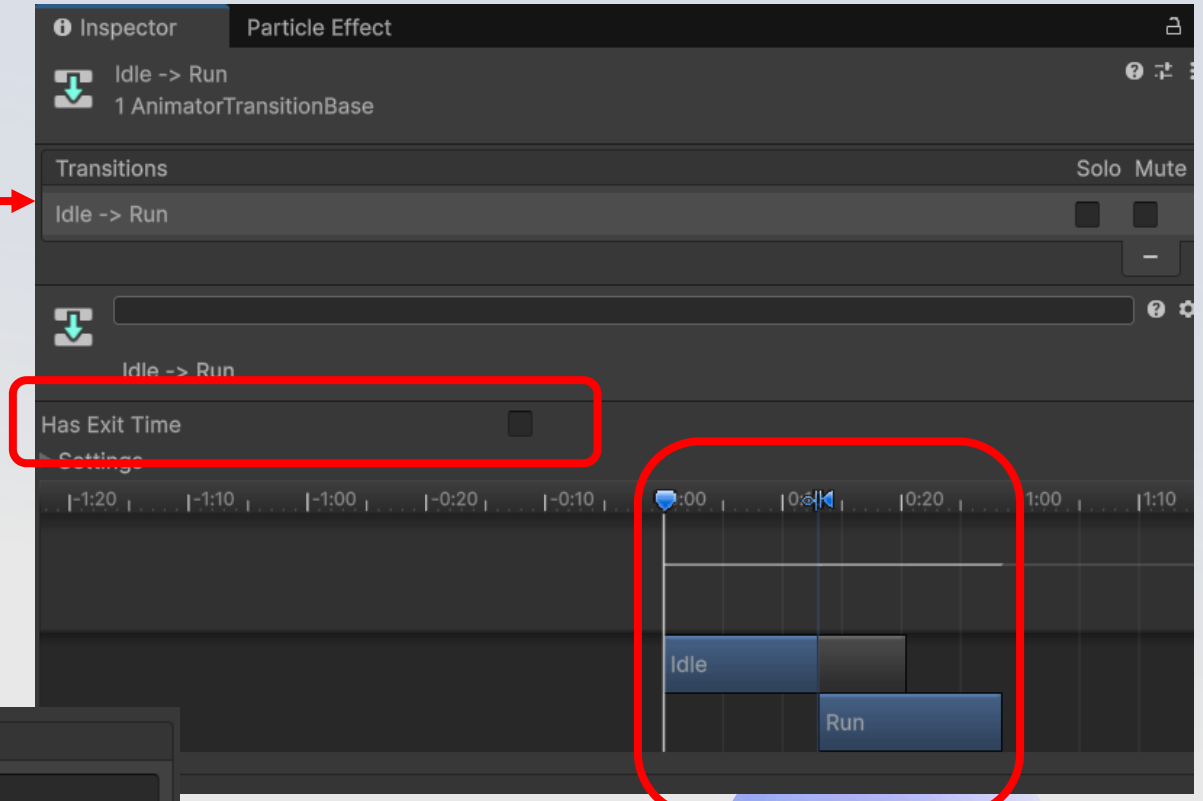
    input = GetComponent<PlayerInput>();
}
```

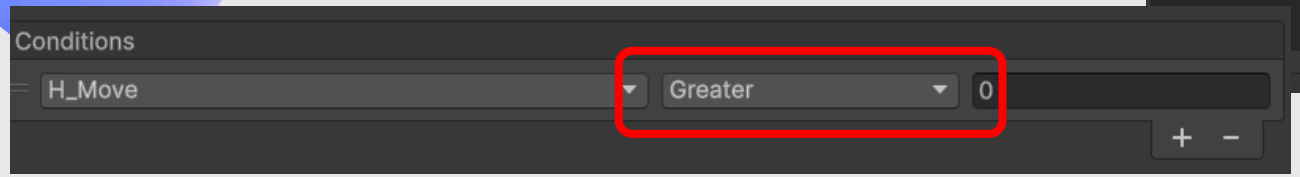
```
private void Animate()
{
    //Move Horizontal
    anim.SetFloat(name: "H_Move", currentMovementSpeed);
}
```

```
private void LateUpdate()
{
    Animate();
}
```

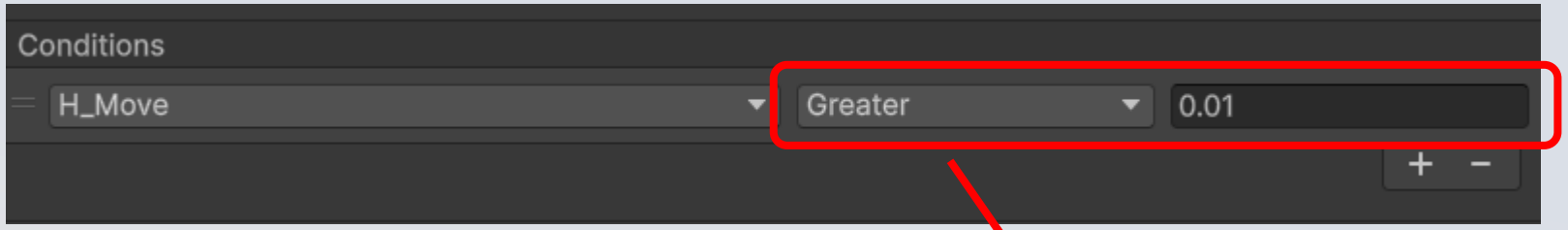
Step 5 – States and Transitions

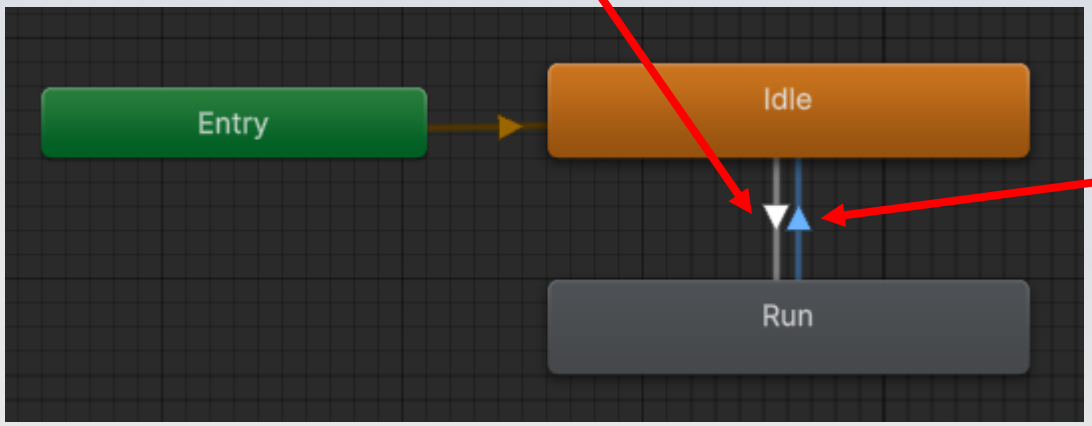






Step 6 – Fix Idle/Run





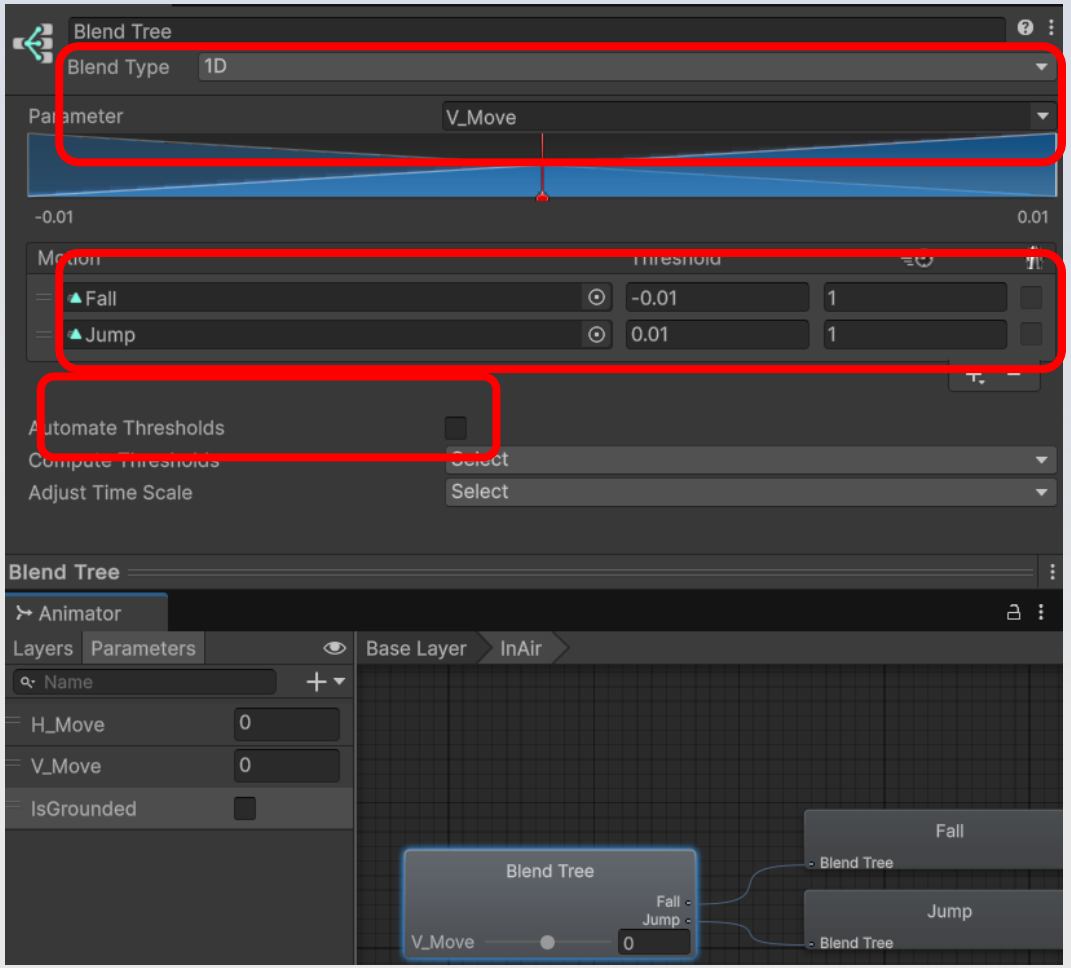



```
private void Animate()
{
    //Move Horizontal
    anim.SetFloat( name: "H_Move",  value: Mathf.Abs(currentMovementSpeed));
}
```

Step 7 – Fix direzione Run

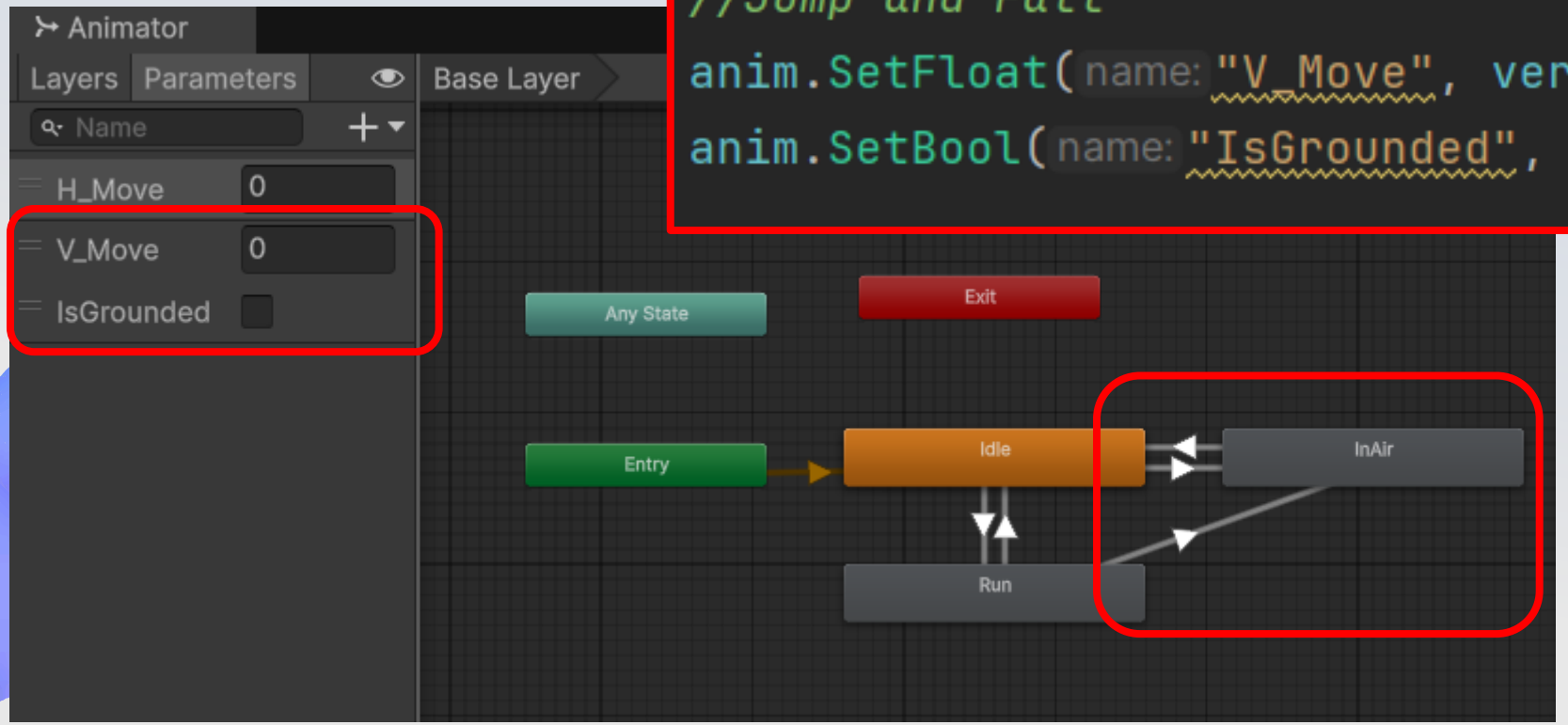
```
private void Animate()
{
    //Move Horizontal
    anim.SetFloat(name: "H_Move", value: Mathf.Abs(currentMovementSpeed));
    if (currentMovementSpeed < 0f)
        spriteRenderer.flipX = true;
    else if (currentMovementSpeed > 0f)
        spriteRenderer.flipX = false;
}
```

Step 8 – Jump/Fall (Blend Trees)



Step 9 – InAir Animation State

```
//Jump and Fall
anim.SetFloat(name: "V_Move", verticalSpeed);
anim.SetBool(name: "IsGrounded", isGrounded);
```



Step 10 – Hit Ceiling Event

Player.cs

```
OnHurt += AnimateHurt;
```

```
new *
public event Action OnHurt;
```

```
private void AnimateHurt()
{
    anim.SetTrigger(name: "TrigHurt");
}
```

```
private void CheckCeiling()
{
    if (isGrounded || verticalSpeed < 0f
        return;

    RaycastHit hitInfo;
    bool ceilingCollision = Physics.Rayc

    if (ceilingCollision)
    {
        verticalSpeed /= 2f;
        float yPoint = hitInfo.point.y -

        transform.position = new Vector3

        OnHurt.Invoke();
    }
}
```

AudioManager.cs

```
private void Start()
{
    FindObjectsByType<Player>(FindObjectsSortMode.None)[0].OnHurt += PlayBonk;
}

Frequently called  1 usage
private void PlayBonk()
{
    source.Play();
}
```

Step 11 – Bonk SFX

